PHASE 7 — DEEP DIVE

Data Engineering for AI

ETL Pipelines · Prefect Advanced · Apache Airflow · dbt · Feature Stores · DVC · Great Expectations

Month 21–22 · Every resource with exact video title + channel name

Phase Duration	8 weeks (Month 21 and 22 of the full roadmap)
Daily Commitment	3–4 hours weekdays, 5–6 hours weekends
Your Starting Point	Phases 1–6 complete — you have a full AI SaaS product with self-hosted inference. Now you need to industrialise the data that feeds it.
The Core Problem	Your AI system is only as good as the data flowing into it. Phase 7 is about making data pipelines reliable, testable, versioned, and observable — the same engineering standards you apply to code.
What Changes	You stop treating data as a static file you loaded once. You build systems that continuously ingest, clean, validate, transform, and serve data to your AI models with full lineage and observability.
Total Topics	8 core topics — ETL fundamentals, Prefect advanced, Apache Airflow, dbt, Pandas advanced pipelines, Feature stores (Feast), DVC, Great Expectations
Who This Is For	Any AI engineer whose models degrade over time because the data pipeline is unreliable — which is every AI engineer in production.
Capstone Project	Production AI Data Platform — Portfolio Project #7

⍰ Phase 7 Philosophy — Data Engineering is AI Engineering

Every AI model failure has a data failure somewhere upstream. A RAG system that returns wrong answers might have stale documents. An LLM that hallucinates might be using poorly chunked text. A fine-tuned model that underperforms might have been trained on uncleaned data. Phase 7 fixes the data layer — making it reliable, tested, and versioned.

AI problem: RAG answers are stale	Data solution: automated ingestion pipeline that detects new/updated documents and re-embeds them within 1 hour of change
AI problem: Model accuracy degrades over time	Data solution: feature store with point-in-time correct feature serving — no data leakage between training and serving
AI problem: Fine-tuning produces worse model	Data solution: DVC-versioned datasets with automated quality checks — impossible to train on bad data
AI problem: 'Where did this answer come from?'	Data solution: full data lineage from source document → parse → chunk → embed → index — every step traceable
AI problem: Pipeline fails silently at 3am	Data solution: Great Expectations data quality checks block bad data from entering the pipeline + Slack alerts
AI problem: Cannot reproduce last month's model	Data solution: DVC tags dataset versions — any model can be reproduced with 'dvc checkout dataset-v1.4'
AI problem: Expensive to add a new data source	Data solution: dbt modular transformation layer — add a new source in one SQL file, lineage auto-generated

TOPIC 1 — ETL / ELT Fundamentals for AI Engineers · *Week 1 · The conceptual foundation before the tools*

ETL (Extract, Transform, Load) and ELT (Extract, Load, Transform) are the patterns that move data from source systems into your AI platform. Before you can use Airflow or dbt, you need a clear mental model of data pipelines, their failure modes, and the design principles that make them reliable.

1.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	ETL vs ELT — What's the Difference and When to Use Each	Zach Wilson (Data with Zach)	YouTube → Search 'ETL vs ELT Difference When to Use Zach Wilson'	Zach Wilson is the best data engineering YouTuber. Essential conceptual grounding.
2	Video	Data Engineering Roadmap 2024 — What You Need to Learn	Zach Wilson (Data with Zach)	YouTube → Search 'Data Engineering Roadmap 2024 What You Need Learn Zach Wilson'	Full data engineering landscape — where Phase 7 tools fit in the bigger picture.
3	Video	Data Pipeline Design Patterns — Batch vs Streaming vs Micro-batch	Zach Wilson (Data with Zach)	YouTube → Search 'Data Pipeline Design Patterns Batch Streaming Micro-batch Zach Wilson'	Three pipeline modes explained with real AI use cases — know which to use when.
4	Video	Idempotent Data Pipelines — The Most Important Data Engineering Concept	Zach Wilson (Data with Zach)	YouTube → Search 'Idempotent Data Pipelines Most Important Data Engineering Concept Zach Wilson'	Idempotency: run the pipeline 10 times, get the same result. Non-negotiable for reliability.
5	Video	Data Quality in Pipelines — Validation, Monitoring, and Alerting	Seattle Data Guy	YouTube → Search 'Data Quality Pipelines Validation Monitoring Alerting Seattle Data Guy'	Introduction to data quality testing — sets up the Great Expectations topic in Topic 8.
6	Video	Change Data Capture (CDC) — Real-Time Data Ingestion Explained	Fivetran (official channel)	YouTube → Fivetran YouTube → Search 'Change Data Capture CDC Real-Time Data Ingestion Explained'	CDC: capture every database change (INSERT/UPDATE/DELETE) and stream it to your pipeline.
7	Video	Data Warehouse vs Data Lake vs Lakehouse — Which Architecture for AI	Zach Wilson (Data with Zach)	YouTube → Search 'Data Warehouse Data Lake Lakehouse Architecture AI Zach Wilson'	Storage architecture for AI data: understand the trade-offs before choosing a stack.
8	Video	Apache Parquet Explained — Why AI Pipelines Use Parquet Instead of CSV	Kahan Data Solutions	YouTube → Search 'Apache Parquet Explained AI Pipelines CSV Kahan Data Solutions'	Parquet is the standard format for ML datasets — columnar, compressed, fast. Know it cold.

1.2 — Core Concepts Every AI Engineer Must Know

ETL vs ELT — The AI Engineer's Version

ETL (traditional)	Extract from source → Transform in Python/Spark → Load into warehouse. Transform before storing. Best when storage is expensive or transformations are complex.
ELT (modern, cloud)	Extract → Load raw data into warehouse/lake → Transform using SQL (dbt). Store raw, transform on query. Best with cheap cloud storage (S3, BigQuery).
For AI pipelines (hybrid)	Extract raw documents → Load to object storage (S3/R2) → Transform: parse + chunk + embed → Load embeddings to Qdrant. Mix of both patterns.

The 4 Pipeline Reliability Principles

- Idempotency: running the pipeline multiple times produces the same result — no duplicates, no missing data
- Atomicity: a pipeline step either completes fully or not at all — no partial writes
- Observability: every pipeline run produces structured logs and metrics — failures are alertable
- Recoverability: a failed pipeline can resume from the last checkpoint — no full rerun needed

Batch vs Streaming vs Micro-batch — When to Use Each

Mode	Pattern	Latency	Best AI Use Case
Batch	Process all data once per day/hour	Minutes to hours	Daily document re-embedding, overnight fine-tuning dataset prep
Micro-batch	Process small batches every 30–60 seconds	30–60 seconds	Near-real-time RAG index updates when documents change
Streaming	Process every event as it arrives	<1 second	Real-time conversation memory, live data ingestion into RAG

For most AI SaaS applications, micro-batch is the right choice. True streaming adds complexity without enough benefit — most users tolerate a 1-minute delay for new documents to appear in search. Batch is fine for training data. Micro-batch for RAG index freshness.

1.3 — Stop Conditions

- ☐ Explain ETL vs ELT and give a concrete AI pipeline example of each
- ☐ Write an idempotent document ingestion function: calling it twice does not create duplicate embeddings
- ☐ Explain the difference between batch, micro-batch, and streaming — name a specific AI use case for each
- ☐ Convert a 1GB CSV dataset to Parquet — compare file size, read speed, and memory usage

- [ ] Design (on paper): a CDC pipeline that detects new rows in PostgreSQL and triggers re-embedding

TOPIC 2 — Prefect Advanced: Production Pipeline Patterns · *Week 1–2 · Beyond Phase 3 basics — enterprise orchestration*

You used Prefect basics in Phase 3. Phase 7 goes deeper: deployments with infrastructure-as-code, event-driven flows that react to S3 uploads or webhook events, work pools for multi-environment execution, and Prefect's integration with the full data engineering stack. This is how you run Prefect in a real production environment.

2.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Course	Prefect 3.0 — Full Tutorial Series (Official, 6 parts)	Prefect (official channel)	YouTube → Prefect official YouTube → Search 'Prefect 3.0 Full Tutorial Series' → open playlist	Official Prefect 3.0 course. Covers Work Pools, Deployments, Events, Automations. Watch all 6.
2	Video	Prefect Work Pools and Workers — Run Flows on Any Infrastructure	Prefect (official channel)	YouTube → Prefect YouTube → Search 'Prefect Work Pools Workers Run Flows Infrastructure'	Work Pools: run your flows on local process, Docker, Kubernetes, or serverless — same code.
3	Video	Prefect Deployments — Deploy Flows with Infrastructure as Code	Prefect (official channel)	YouTube → Prefect YouTube → Search 'Prefect Deployments Deploy Flows Infrastructure as Code'	Define deployments in code (prefect.yaml) — version-controlled pipeline definitions.
4	Video	Prefect Events and Automations — Event-Driven Flows	Prefect (official channel)	YouTube → Prefect YouTube → Search 'Prefect Events Automations Event-Driven Flows'	Trigger a flow when an S3 file arrives, when a sensor fires, or when another flow completes.
5	Video	Prefect with Docker — Containerised Flow Execution	Zach Wilson (Data with Zach)	YouTube → Search 'Prefect Docker Containerised Flow Execution Zach Wilson'	Run each flow in its own Docker container — dependency isolation for multi-team pipelines.
6	Video	Prefect vs Airflow 2025 — Detailed Comparison	Seattle Data Guy	YouTube → Search 'Prefect vs Airflow 2025 Detailed Comparison Seattle Data Guy'	Deep comparison with practical guidance. Helps decide which to use for specific scenarios.
7	Video	Prefect AI Pipeline — Build an End-to-End ML Data Pipeline	Zach Wilson (Data with Zach)	YouTube → Search 'Prefect AI Pipeline End-to-End ML Data Pipeline Zach Wilson'	Zach builds a complete ML data pipeline with Prefect. The closest real-world analog to your capstone.
8	Video	Prefect Notifications — Slack, PagerDuty, and Custom Webhooks	Prefect (official channel)	YouTube → Prefect YouTube → Search 'Prefect Notifications Slack PagerDuty Custom Webhooks'	Production alert routing: different failures trigger different notification channels.
9	Docs	Prefect 3.0 Documentation — Work Pools, Deployments, Events	Prefect Team	docs.prefect.io — read: Develop + Deploy +	Read the full Develop, Deploy, and Automate

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
				Automate sections completely	sections. Prefect docs are excellent.

2.2 — Advanced Prefect Patterns

Prefect Deployments — Infrastructure as Code

```
# prefect.yaml — version-controlled deployment definition
# This replaces manual UI clicks — deployments defined in code

deployments:
  - name: ai-document-ingestion
    entrypoint: flows/ingestion.py:ingest_documents

    # Run on a Docker worker (isolated environment per run)
    work_pool:
      name: docker-pool
      job_variables:
        image: your-registry/ai-pipeline:latest
        env:
          OPENAI_API_KEY: '{{ prefect.blocks.secret.openai-key }}'
          QDRANT_URL: '{{ prefect.blocks.secret.qdrant-url }}'

    # Schedule: run every 30 minutes
    schedules:
      - cron: '*/30 * * * *'
        timezone: Asia/Kolkata

    # Parameters passed to the flow function
    parameters:
      source_bucket: 'ai-documents-prod'
      max_documents_per_run: 500

# Deploy: prefect deploy --all
# Each environment (dev/staging/prod) has its own prefect.yaml
```

Event-Driven Flows — React to Real-World Events

- S3 trigger: new file uploaded to S3 → Prefect Automation detects S3 event → triggers ingestion flow
- Webhook trigger: GitHub push event → triggers documentation ingestion flow
- Flow dependency: flow-B starts when flow-A completes successfully
- Custom sensor: Prefect polls an API endpoint every 5 minutes — triggers when new data available
- Schedule + event combo: schedule triggers backup; event triggers urgent re-embedding

Prefect Concurrency and Rate Limiting

```
from prefect import flow, task
from prefect.concurrency.sync import concurrency

# Global concurrency limit — prevent overloading downstream services
@task(retries=3, retry_delay_seconds=exponential_backoff(backoff_factor=2))
async def embed_chunk(chunk: str, chunk_id: str) -> list[float]:
    # Limit to 20 concurrent OpenAI embedding calls
    async with concurrency('openai-embeddings', occupy=1):
        response = await openai.embeddings.create(
            input=chunk,
            model='text-embedding-3-small'
        )
        return response.data[0].embedding

# Set the limit in Prefect Cloud UI or via CLI:
# prefect concurrency-limit create openai-embeddings --limit 20

# Result: no matter how many workers run in parallel,
# never more than 20 concurrent OpenAI calls globally
```

2.3 — Stop Conditions

- ☐ Prefect 3.0 deployment defined in prefect.yaml — deploy with 'prefect deploy --all'
- ☐ Work pool: Docker worker pool running — flows execute in isolated containers
- ☐ Event-driven trigger: new file in S3 bucket automatically triggers ingestion flow
- ☐ Concurrency limit: 'openai-embeddings' limit set to 20 — never exceeded across all workers
- ☐ Multi-environment: separate dev/staging/prod deployments with different parameters from prefect.yaml
- ☐ Alert routing: pipeline failure → Slack #data-alerts; SLA breach → PagerDuty
- ☐ Flow run history: 30-day dashboard shows all runs, durations, and failure rates

TOPIC 3 — Apache Airflow: Enterprise Pipeline Orchestration · *Weeks 2–3 · The industry standard for data pipelines*

Apache Airflow is the most widely deployed pipeline orchestrator in production data engineering teams. While Prefect is simpler and more developer-friendly, Airflow is the tool you will find in most enterprise data platforms. Understanding Airflow is essential for working in any data-heavy company — and many AI teams run their training and ingestion pipelines on Airflow.

3.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Course	The Complete Hands-On Introduction to Apache Airflow	Marc Lamberti (official Airflow instructor)	YouTube → Search 'Complete Hands-On Introduction Apache Airflow Marc Lamberti' → find full course or playlist	Marc Lamberti is the #1 Airflow educator globally. Start with his free YouTube content.
2	Course	Apache Airflow Fundamentals — Free Course (Astronomer Academy)	Astronomer (official channel)	YouTube → Astronomer YouTube → Search 'Apache Airflow Fundamentals Free Course Astronomer Academy'	Official Astronomer Airflow course. Astronomer is the biggest Airflow platform company.
3	Video	Apache Airflow Tutorial for Beginners — Full Course (2.5 hrs)	CodeWithYu	YouTube → Search 'Apache Airflow Tutorial Beginners Full Course CodeWithYu'	Best beginner Airflow tutorial. Covers DAGs, operators, scheduling, dependencies.
4	Video	Airflow TaskFlow API — Modern Python DAGs Without Operators	Marc Lamberti	YouTube → Search 'Airflow TaskFlow API Modern Python DAGs Without Operators Marc Lamberti'	TaskFlow API is the modern Airflow syntax (@task decorator) — replaces old operator boilerplate.
5	Video	Airflow Dynamic Task Mapping — Parallel Processing at Scale	Marc Lamberti	YouTube → Search 'Airflow Dynamic Task Mapping Parallel Processing Scale Marc Lamberti'	Process 1000 documents in parallel with one DAG — dynamic task mapping explained clearly.
6	Video	Airflow with Docker and Astro CLI — Local Development Setup	Marc Lamberti	YouTube → Search 'Airflow Docker Astro CLI Local Development Setup Marc Lamberti'	Set up Airflow locally in <10 minutes using the Astro CLI — the recommended dev setup.
7	Video	Airflow Sensors — Wait for External Events in Your DAGs	Marc Lamberti	YouTube → Search 'Airflow Sensors Wait External Events DAGs Marc Lamberti'	S3 sensor, HTTP sensor, SQL sensor — wait for external conditions before proceeding.
8	Video	Airflow XComs — Pass Data Between Tasks	Marc Lamberti	YouTube → Search 'Airflow XComs Pass Data Between Tasks Marc Lamberti'	XComs: how tasks communicate data — pass document counts, embeddings status, model metrics.

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
9	Video	Airflow Best Practices — Production-Grade DAGs	Astronomer (official channel)	YouTube → Astronomer YouTube → Search 'Airflow Best Practices Production-Grade DAGs'	Astronomer's production best practices — idempotency, catchup, pool limits, DAG design.
10	Video	Airflow on Kubernetes — KubernetesPodOperator and KubernetesExecutor	Marc Lamberti	YouTube → Search 'Airflow Kubernetes KubernetesPodOperator KubernetesExecutor Marc Lamberti'	Run each Airflow task in its own Kubernetes pod — the enterprise production deployment pattern.
11	Video	Airflow vs Prefect vs Dagster — 2024 Comparison	Seattle Data Guy	YouTube → Search 'Airflow vs Prefect vs Dagster 2024 Comparison Seattle Data Guy'	Comprehensive tool comparison. Seattle Data Guy is objective and thorough.
12	Docs	Apache Airflow Official Documentation	Apache Software Foundation	airflow.apache.org/docs — read: Tutorial on the Taskflow API + Best Practices	Read the Taskflow API Tutorial and Best Practices completely before writing DAGs.

3.2 — Core Concepts

DAG — Directed Acyclic Graph

A DAG in Airflow is a Python file that defines a set of tasks and the dependencies between them. 'Directed' means the data flows in one direction. 'Acyclic' means no circular dependencies. 'Graph' means tasks are nodes and dependencies are edges.

- Each DAG file = one pipeline definition
- Scheduler reads DAG files every 30 seconds and queues runs based on schedule
- Each DAG run creates individual task instances — each independently retried on failure
- DAG runs are immutable — a failed run stays failed; you trigger a new run

TaskFlow API — The Modern Airflow Syntax

```
# Modern Airflow with TaskFlow API (@task decorator)
from airflow.decorators import dag, task
from datetime import datetime, timedelta
import pendulum

@dag(
    dag_id='ai_document_pipeline',
    schedule='@hourly',
    start_date=pendulum.datetime(2024, 1, 1, tz='Asia/Kolkata'),
    catchup=False,              # don't run missed schedules on startup
    max_active_runs=1,          # only one run at a time
    default_args={
        'retries': 3,
        'retry_delay': timedelta(minutes=5),
```

```
        'retry_exponential_backoff': True,
    },
    tags=['ai', 'ingestion', 'production'],
)
def ai_document_pipeline():

    @task()  # TaskFlow — no need for PythonOperator wrapper
    def extract_new_documents() -> list[dict]:
        # Fetch documents added/updated in last hour
        return fetch_from_database(since=datetime.now() - timedelta(hours=1))

    @task()                             # Automatically retried on failure
    def parse_documents(docs: list[dict]) -> list[dict]:
        return [parse(doc) for doc in docs]

    @task.expand()                      # Dynamic task mapping — parallel!
    def embed_document(doc: dict) -> dict:
        doc['embedding'] = generate_embedding(doc['text'])
        return doc

    @task()                             # Bulk upsert after all embeddings done
    def store_in_qdrant(embedded_docs: list[dict]):
        qdrant_client.upsert(collection='documents', points=embedded_docs)

    # Task dependencies defined by function calls — clear and Pythonic
    raw  = extract_new_documents()
    parsed = parse_documents(raw)
    embedded = embed_document.expand(doc=parsed)  # one task per document
    store_in_qdrant(embedded)

dag = ai_document_pipeline()
```

Production DAG Patterns for AI Pipelines

- Catchup=False: do NOT reprocess historical data on pipeline restart — AI pipelines are expensive
- max_active_runs=1: prevent concurrent runs of the same pipeline — avoids duplicate embeddings
- Pools: limit concurrent database connections or API calls globally across all DAGs
- Dynamic task mapping: @task.expand() — process each document as a separate retryable task
- SLA miss alerts: if a pipeline doesn't complete in 2 hours, alert the team
- Trigger rules: ALL_SUCCESS (default), ONE_FAILED, NONE_FAILED_MIN_ONE_SUCCESS

> ⍰ The most important Airflow best practice for AI pipelines: set catchup=False and max_active_runs=1. Without these, a deployment restart will attempt to run every missed schedule (potentially thousands of runs) and multiple pipelines will compete to embed the same documents.

3.3 — Stop Conditions

☐ Airflow running locally via Astro CLI — UI accessible at localhost:8080
☐ Write a TaskFlow DAG with 4 tasks: extract → parse → embed → store — all using @task decorator
☐ Dynamic task mapping: @task.expand() processes each document as a separate task — 100 docs = 100 tasks
☐ S3 sensor: DAG waits for a file in S3 before starting processing — tested with file upload
☐ Pool: 'openai-api' pool with 20 slots — no more than 20 concurrent embedding tasks globally
☐ SLA miss: alert fires when the hourly pipeline hasn't completed within 90 minutes
☐ XCom: task 1 passes document count to task 4 which logs 'Processed N documents'
☐ Production DAG: catchup=False, max_active_runs=1, retries=3, retry_exponential_backoff=True

TOPIC 4 — dbt (Data Build Tool): SQL Transformations with Engineering Standards

Week 3–4 · Version control, testing, and lineage for your SQL

dbt (data build tool) brings software engineering practices to SQL transformations. Before dbt, SQL transformations were scattered scripts with no versioning, no testing, and no documentation. dbt makes SQL transformations modular, testable, version-controlled, and self-documenting. It is the most widely adopted data transformation tool in the modern data stack.

4.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Course	dbt (Data Build Tool) — Complete Beginner to Advanced Course	dbt Labs (official channel)	YouTube → dbt Labs official YouTube → Search 'dbt Complete Beginner Advanced Course' → playlist	Official dbt tutorial series from the creators. Watch the full beginner series first.
2	Course	dbt Tutorial — Full Course for Data Engineers (3.5 hrs)	Kahan Data Solutions	YouTube → Search 'dbt Tutorial Full Course Data Engineers Kahan Data Solutions'	Kahan Data Solutions has the best standalone dbt YouTube course. Follow along completely.
3	Video	dbt Core Tutorial — Install, Configure, and Run Your First Models	Seattle Data Guy	YouTube → Search 'dbt Core Tutorial Install Configure Run First Models Seattle Data Guy'	Hands-on dbt Core setup — runs against a local PostgreSQL database.
4	Video	dbt Models, Sources, and References — Building Modular SQL	dbt Labs (official channel)	YouTube → dbt Labs YouTube → Search 'dbt Models Sources References Building Modular SQL'	Core dbt concepts: models, sources, ref(), staging → intermediate → mart pattern.
5	Video	dbt Tests — Data Quality Built Into Your Transformations	Kahan Data Solutions	YouTube → Search 'dbt Tests Data Quality Built Into Transformations Kahan Data Solutions'	Built-in tests (not_null, unique, accepted_values) + custom tests. Essential for data quality.
6	Video	dbt Documentation — Auto-Generate Data Catalog	dbt Labs (official channel)	YouTube → dbt Labs YouTube → Search 'dbt Documentation Auto-Generate Data Catalog'	dbt docs generate: automatic data catalog with column descriptions, lineage graph, test results.
7	Video	dbt Incremental Models — Process Only New Data	Seattle Data Guy	YouTube → Search 'dbt Incremental Models Process Only New Data Seattle Data Guy'	Incremental models: process only new/changed rows — essential for large AI training datasets.
8	Video	dbt Macros and Jinja — Reusable SQL Logic	Kahan Data Solutions	YouTube → Search 'dbt Macros Jinja Reusable SQL Logic Kahan Data Solutions'	Macros: write SQL helper functions once, use in every model. Jinja templating in SQL.
9	Video	dbt with Airflow — Orchestrate dbt Runs in Airflow DAGs	Marc Lamberti	YouTube → Search 'dbt with Airflow Orchestrate dbt Runs	Combine dbt (transformations) with Airflow (orchestration) —

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
				Airflow DAGs Marc Lamberti'	the standard production pattern.
10	Video	dbt Lineage — Data Lineage Graph and Impact Analysis	dbt Labs (official channel)	YouTube → dbt Labs YouTube → Search 'dbt Lineage Data Lineage Graph Impact Analysis'	Lineage graph: visualise how every table depends on every other table. Trace data problems upstream.
11	Docs	dbt Core Documentation — Getting Started Project	dbt Labs Team	docs.getdbt.com — follow the Quickstart Guide for PostgreSQL completely	The dbt quickstart is excellent. Do it before any other dbt work — takes 30 minutes.

4.2 — dbt for AI Pipelines

dbt Project Structure for an AI Data Platform

```
ai_data_platform/           # dbt project root
├── dbt_project.yml         # Project config: database, schema prefixes
├── profiles.yml            # Database connection (PostgreSQL/BigQuery/Snowflake)
│
├── models/
│   ├── staging/            # 1:1 mapping of source tables, minimal transformation
│   │   ├── stg_documents.sql         # raw documents from API/upload
│   │   ├── stg_document_chunks.sql   # raw chunks from parsing pipeline
│   │   └── stg_llm_api_calls.sql     # raw LLM API call logs
│   │
│   ├── intermediate/       # Join, aggregate, apply business rules
│   │   ├── int_document_chunks_enriched.sql  # chunks + document metadata
│   │   └── int_api_cost_by_model.sql          # cost per model per day
│   │
│   └── marts/              # Final tables — consumed by application and analytics
│       ├── ai/
│       │   ├── fct_rag_queries.sql    # RAG query fact table for analytics
│       │   └── fct_embedding_jobs.sql # embedding pipeline runs
│       └── finance/
│           └── fct_ai_cost_daily.sql  # daily LLM cost per user, model, feature
│
├── tests/                  # Custom data tests (SQL files)
│   └── assert_no_orphan_chunks.sql  # all chunks reference a valid document
│
├── macros/                 # Reusable Jinja SQL functions
│   └── calculate_token_cost.sql    # macro: input tokens × rate + output tokens ×
rate
│
└── seeds/                  # Static reference data
    └── model_pricing.csv   # GPT-4o: $2.50/M input, $10.00/M output
```

dbt Incremental Models — Critical for AI Datasets

```
-- models/marts/ai/fct_embedding_jobs.sql
-- Incremental model: only process new embedding jobs since last run

{{
  config(
    materialized='incremental',
    unique_key='job_id',
    on_schema_change='append_new_columns',
  )
}}

SELECT
    j.job_id,
    j.document_id,
    j.model_name,
    j.input_tokens,
    j.output_tokens,
    j.cost_usd,
    j.duration_ms,
    j.created_at,
    d.title        AS document_title,
    d.org_id
FROM {{ ref('stg_documents') }} d
JOIN {{ ref('stg_embedding_jobs') }} j USING (document_id)

{% if is_incremental() %}
  -- Only process rows added since the last dbt run
  WHERE j.created_at > (SELECT MAX(created_at) FROM {{ this }})
{% endif %}

-- 'dbt run --select fct_embedding_jobs'
-- First run: processes ALL historical data
-- Subsequent runs: only processes NEW rows — fast and cheap
```

dbt Tests — Data Quality as Code

```
# models/staging/schema.yml — define tests declaratively
models:
  - name: stg_documents
    columns:
      - name: document_id
        tests:
          - not_null
          - unique                        # no duplicate document IDs
      - name: org_id
```

```
        tests:
          - not_null
          - relationships:
              to: ref('stg_organisations')
              field: org_id               # every doc has a valid org
      - name: status
        tests:
          - accepted_values:
              values: ['pending', 'processing', 'embedded', 'failed']
      - name: token_count
        tests:
          - dbt_utils.accepted_range:
              min_value: 1
              max_value: 100000           # flag suspiciously large docs

# Run tests: dbt test
# All failures are shown in the output — block pipeline if tests fail
```

4.3 — Stop Conditions

- ☐ dbt project initialised against your PostgreSQL database — 'dbt debug' passes
- ☐ 3-layer model structure: staging → intermediate → marts — at least 2 models per layer
- ☐ Incremental model: fct_embedding_jobs only processes new rows on each run — verified with row counts
- ☐ dbt tests: not_null + unique + relationships tests pass on all staging models
- ☐ dbt docs: 'dbt docs generate && dbt docs serve' — lineage graph visible in browser
- ☐ Macro: calculate_token_cost() macro used in 3 different models — no copy-paste SQL
- ☐ Airflow integration: a DAG triggers 'dbt run --select marts' after the ingestion pipeline completes
- ☐ Schema tests block pipeline: failed dbt test prevents downstream tasks from running

TOPIC 5 — Advanced Pandas Pipelines & Polars · *Week 4 · High-performance Python data processing for AI datasets*

Phase 3 covered Pandas basics. Phase 7 goes deeper: method chaining for pipeline-style transformations, Polars for 5–50x faster processing on large datasets, and optimised patterns for the specific data shapes that AI pipelines work with — document corpora, embedding matrices, and token usage logs.

5.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	Pandas Method Chaining — Write Cleaner Pipeline Code	Matt Harrison	YouTube → Search 'Pandas Method Chaining Write Cleaner Pipeline Code Matt Harrison'	Method chaining transforms messy Pandas code into readable, testable pipelines.
2	Video	Polars Tutorial — The Fastest DataFrame Library in Python	Liam Brannigan	YouTube → Search 'Polars Tutorial Fastest DataFrame Library Python Liam Brannigan'	Best Polars tutorial. Liam Brannigan is the top Polars educator on YouTube.
3	Video	Polars vs Pandas — 20x Performance Comparison	Rob Mulla	YouTube → Search 'Polars vs Pandas 20x Performance Comparison Rob Mulla'	Direct benchmark: Polars is 5–50x faster than Pandas on most operations.
4	Video	Polars Lazy API — Process Data Larger Than RAM	Liam Brannigan	YouTube → Search 'Polars Lazy API Process Data Larger Than RAM Liam Brannigan'	Polars lazy evaluation: build a query plan, execute it optimally. Process 100GB files.
5	Video	Pandas Pipe — Build Composable Data Transformation Pipelines	Matt Harrison	YouTube → Search 'Pandas Pipe Build Composable Data Transformation Pipelines Matt Harrison'	pandas .pipe() method — compose transformation functions into clean, testable pipelines.
6	Video	Efficient Pandas — Memory Optimisation and Chunked Processing	Rob Mulla	YouTube → Search 'Efficient Pandas Memory Optimisation Chunked Processing Rob Mulla'	Reduce Pandas memory by 70% with dtype optimisation — process AI datasets that don't fit in RAM.
7	Video	Polars with Pydantic — Type-Safe Data Pipelines	Liam Brannigan	YouTube → Search 'Polars Pydantic Type-Safe Data Pipelines Liam Brannigan'	Validate Polars DataFrame schemas with Pydantic — prevents type errors in production pipelines.

5.2 — Key Patterns

Method Chaining — The Pipeline Pattern

```
# BAD: imperative, hard to test, many intermediate variables
df = pd.read_parquet('documents.parquet')
df = df.dropna(subset=['content', 'doc_id'])
df = df[df['token_count'] < 8000]
df['content'] = df['content'].str.strip().str.lower()
df = df.drop_duplicates(subset=['doc_id'])
df = df.reset_index(drop=True)

# GOOD: method chaining — one expression, readable left to right
df = (
    pd.read_parquet('documents.parquet')
    .dropna(subset=['content', 'doc_id'])
    .query('token_count < 8000')       # readable filter
    .assign(content=lambda d: d['content'].str.strip().str.lower())
    .drop_duplicates(subset=['doc_id'])
    .reset_index(drop=True)
)

# BEST: composable named functions with .pipe()
def remove_invalid_docs(df): return df.dropna(subset=['content', 'doc_id'])
def filter_long_docs(df, max_tokens=8000): return df.query(f'token_count <
{max_tokens}')
def normalise_text(df): return
df.assign(content=df['content'].str.strip().str.lower())
def deduplicate(df): return
df.drop_duplicates(subset=['doc_id']).reset_index(drop=True)

df = (
    pd.read_parquet('documents.parquet')
    .pipe(remove_invalid_docs)
    .pipe(filter_long_docs, max_tokens=8000)
    .pipe(normalise_text)
    .pipe(deduplicate)
)
# Each function is independently unit-testable
```

Polars — When to Switch from Pandas

- File size >500MB: Polars is 5–50x faster than Pandas on groupby, join, filter
- RAM limited: Polars lazy API processes files larger than RAM — Pandas crashes
- Parallel processing: Polars uses all CPU cores automatically — Pandas is single-threaded
- Type safety: Polars enforces column types strictly — prevents silent type coercion bugs
- Streaming mode: process 100GB Parquet files row by row — never loads full file into RAM

> ⍰ Rule of thumb: use Pandas for datasets under 500MB (quick iteration, familiar API). Switch to Polars for datasets over 500MB or when processing speed is a bottleneck. The Polars API is 80% similar to Pandas — the learning curve is 1–2 days.

5.3 — Stop Conditions

☐ Rewrite a 5-step Pandas pipeline using method chaining + .pipe() — each step independently testable
☐ Polars: process a 1GB Parquet file — confirm it runs 10x+ faster than Pandas equivalent
☐ Polars lazy mode: process a 10GB file without loading it fully into RAM — no MemoryError
☐ Schema validation: Pydantic or Polars schema enforces column types — invalid data raises before pipeline runs
☐ Chunked Pandas: process a 5GB CSV with pd.read_csv(chunksize=10000) — no OOM, correct final result

TOPIC 6 — Feature Stores with Feast · *Week 5 · Serve consistent features between training and production*

A feature store solves the most subtle and expensive problem in ML: training-serving skew. Features computed in Python during training must exactly match features computed in production at inference time. Without a feature store, this is nearly impossible to guarantee. Feast is the most widely used open-source feature store.

6.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	Feature Stores Explained — What They Are and Why You Need One	Weights & Biases (official channel)	YouTube → W&B YouTube → Search 'Feature Stores Explained What They Are Why You Need'	Best conceptual explanation of the feature store problem before diving into Feast.
2	Video	Feast Feature Store — Complete Tutorial	Feast (official channel)	YouTube → Feast official YouTube → Search 'Feast Feature Store Complete Tutorial'	Official Feast tutorial. Covers feature definitions, materialization, and retrieval.
3	Video	Feast Getting Started — Feature Engineering for ML in Production	Tecton (official channel)	YouTube → Tecton YouTube → Search 'Feast Getting Started Feature Engineering ML Production'	Tecton (Feast creators' company) explains Feast concepts and production patterns.
4	Video	Point-in-Time Correct Features — Preventing Data Leakage in ML	Chip Huyen (Author of Designing ML Systems)	YouTube → Search 'Point-in-Time Correct Features Preventing Data Leakage ML Chip Huyen'	The most important feature store concept. Chip Huyen is the definitive ML systems educator.
5	Video	Feature Stores for LLMs — How Embeddings Fit Into the Feature Store	Zach Wilson (Data with Zach)	YouTube → Search 'Feature Stores LLMs Embeddings Fit Feature Store Zach Wilson'	How to use Feast with LLM-based features including embeddings and document metadata.
6	Video	Feast Redis Online Store — Low-Latency Feature Serving	Feast (official channel)	YouTube → Feast YouTube → Search 'Feast Redis Online Store Low-Latency Feature Serving'	Redis as online store: serve features in <10ms at inference time — production setup.
7	Book	Designing Machine Learning Systems — Chapter 5: Feature Engineering (book)	Chip Huyen	oreilly.com → Designing Machine Learning Systems — Chapter 5 (or available summary on chip.huyen.ai)	The canonical ML systems textbook. Chapter 5 on features is must-read for AI engineers.

6.2 — Core Concepts

The Training-Serving Skew Problem

Training: you compute 'user's average session duration in the last 7 days' using a Python script over historical data. Serving: you compute the same feature using a different code path in your FastAPI service. Even small differences (timezone handling, NULL values, rounding) produce different values — the model performs worse in production than in evaluation. This is training-serving skew.

- Feast solution: define features ONCE in Python, materialise to offline store (Parquet/BigQuery) for training and online store (Redis) for serving
- Same feature definition, same values, both environments — skew eliminated

Point-in-Time Correct Features

- Problem: when training, you must use only features that were available BEFORE the training label's timestamp
- Example: predicting document quality at time T — you cannot use features computed at T+1
- Feast get_historical_features(): automatically performs point-in-time join
- Without this: data leakage — model trains on future information, appears to perform well, fails in production

Feast Architecture for an AI Platform

```
# feature_repo/features.py — define features once
from feast import Entity, FeatureView, Field, FileSource
from feast.types import Float32, Int64, String
from datetime import timedelta

# Entity — the thing we compute features for
document = Entity(name='document_id', value_type=String)

# Feature source — where raw features come from (dbt output)
document_features_source = FileSource(
    path='s3://ai-platform/features/document_features.parquet',
    timestamp_field='created_at',
)

# Feature view — defines which features to serve
document_feature_view = FeatureView(
    name='document_features',
    entities=[document],
    ttl=timedelta(days=30),          # features expire after 30 days
    schema=[
        Field(name='token_count',     dtype=Int64),
        Field(name='avg_chunk_score', dtype=Float32),
        Field(name='embedding_model', dtype=String),
        Field(name='language',        dtype=String),
    ],
    source=document_features_source,
    online=True,                     # materialise to Redis for serving
)

# Materialise offline → online (run on schedule)
# feast materialize-incremental $(date -u +"%Y-%m-%dT%H:%M:%S")
```

```
# Retrieve features at inference time (<10ms)
store = FeatureStore(repo_path='.')
features = store.get_online_features(
    features=['document_features:token_count', 'document_features:language'],
    entity_rows=[{'document_id': 'doc-123'}],
).to_dict()
```

6.3 — Stop Conditions

- ☐ Feast project initialised — 'feast apply' creates feature tables in the registry
- ☐ Feature view defined with at least 4 features from your AI platform's document data
- ☐ Historical features: retrieve point-in-time correct features for a training dataset
- ☐ Online features: Redis online store materialised — feature retrieval latency <10ms confirmed
- ☐ Scheduled materialisation: Airflow DAG or Prefect flow runs 'feast materialize-incremental' hourly
- ☐ Training-serving parity: same document_id returns identical feature values from offline and online store
- ☐ Explain point-in-time join: demonstrate how Feast prevents data leakage in a training scenario

TOPIC 7 — DVC: Data Version Control · *Week 6 · Git for datasets and ML models — reproducibility guaranteed*

DVC (Data Version Control) is Git for data and ML models. It tracks large files (datasets, model weights, embeddings) in cloud storage (S3, GCS, Azure) while storing only lightweight metadata in Git. Every dataset version is reproducible, every experiment is tracked, and every model can be traced back to the exact data it was trained on.

7.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Course	DVC — Official Tutorial Series (Complete, 8 videos)	DVC (official channel)	YouTube → DVC official YouTube → Search 'DVC Official Tutorial Series Complete' → open playlist	Official DVC tutorial series from the creators. Watch all 8 videos — the complete foundation.
2	Video	DVC Tutorial — Data Version Control for Machine Learning	Krish Naik	YouTube → Search 'DVC Tutorial Data Version Control Machine Learning Krish Naik'	Krish Naik's practical DVC tutorial — installs DVC and tracks a real dataset from scratch.
3	Video	DVC with Git — Track Data, Models, and Experiments Together	DVC (official channel)	YouTube → DVC YouTube → Search 'DVC Git Track Data Models Experiments Together'	DVC + Git workflow: how .dvc files work, remote storage setup, checkout and push.
4	Video	DVC Pipelines — Reproducible ML Workflows	DVC (official channel)	YouTube → DVC YouTube → Search 'DVC Pipelines Reproducible ML Workflows'	DVC pipeline stages — dependency graph ensures only changed stages re-run. Like Make for ML.
5	Video	DVC Remote Storage — S3, Google Cloud, and Azure	DVC (official channel)	YouTube → DVC YouTube → Search 'DVC Remote Storage S3 Google Cloud Azure'	Configure DVC remote — store datasets and model weights in S3, any team member can pull.
6	Video	DVC Experiments — Track and Compare ML Experiment Results	DVC (official channel)	YouTube → DVC YouTube → Search 'DVC Experiments Track Compare ML Experiment Results'	DVC experiments: track hyperparameters, metrics, and plots — compare across runs.
7	Video	MLflow vs DVC — Which ML Tracking Tool to Use	Krish Naik	YouTube → Search 'MLflow vs DVC Which ML Tracking Tool Use Krish Naik'	DVC (data + pipeline versioning) vs MLflow (experiment tracking) — use both together.
8	Docs	DVC Documentation — Get Started Guide	DVC Team	dvc.org/doc — complete the Get Started guide (30 minutes)	The DVC Get Started guide is hands-on and concise. Complete it before watching any videos.

7.2 — DVC Core Patterns

Tracking Datasets and Model Weights

```
# Set up DVC in an existing Git repo
git init && dvc init

# Configure remote storage (S3)
dvc remote add -d s3-remote s3://ai-platform-data/dvc-store
dvc remote modify s3-remote region ap-south-1

# Track a large dataset file with DVC
# (the file itself goes to S3, a tiny .dvc pointer goes to Git)
dvc add data/training_docs.parquet
git add data/training_docs.parquet.dvc .gitignore
git commit -m 'Track training dataset v1.0'
dvc push  # upload actual data to S3

# Any team member can reproduce:
git clone <repo>
dvc pull  # downloads data from S3

# Create a new dataset version
# (update training_docs.parquet with new documents)
dvc add data/training_docs.parquet
git add data/training_docs.parquet.dvc
git commit -m 'Dataset v1.1: added 5000 new documents'
git tag dataset-v1.1
dvc push

# Checkout a specific dataset version
git checkout dataset-v1.0
dvc checkout  # restores the exact v1.0 data from S3
```

DVC Pipelines — Reproducible Processing Stages

```
# dvc.yaml — define pipeline stages with dependencies
# DVC tracks inputs and outputs — only re-runs changed stages
stages:
  parse_documents:
    cmd: python src/parse.py --input data/raw_docs --output data/parsed
    deps:
      - src/parse.py
      - data/raw_docs/            # re-run if raw docs change
    outs:
      - data/parsed/              # output tracked by DVC
```

```
  generate_embeddings:
    cmd: python src/embed.py --input data/parsed --output data/embedded
    deps:
      - src/embed.py
      - data/parsed/              # re-run if parsed docs change
    outs:
      - data/embedded/
    metrics:
      - metrics/embedding_stats.json:  # tracks embedding quality metrics
          cache: false

  train_classifier:
    cmd: python src/train.py --input data/embedded --output models/
    deps:
      - src/train.py
      - data/embedded/
    outs:
      - models/classifier.pkl
    metrics:
      - metrics/eval_results.json:
          cache: false

# Run the full pipeline (only changed stages re-execute):
# dvc repro

# Compare metrics between pipeline runs:
# dvc metrics diff HEAD~1
```

7.3 — Stop Conditions

- ☐ DVC initialised in your Git repo — 'dvc status' shows tracked files
- ☐ S3 remote configured — large dataset file stored in S3, tiny .dvc pointer in Git
- ☐ Dataset versioned: v1.0 and v1.1 tags — 'git checkout dataset-v1.0 && dvc checkout' restores exact data
- ☐ DVC pipeline: dvc.yaml with 3 stages — 'dvc repro' only re-runs stages whose inputs changed
- ☐ Metrics tracked: evaluation metrics stored in metrics/ and visible with 'dvc metrics show'
- ☐ Team workflow: second developer can clone repo + 'dvc pull' and reproduce your full pipeline
- ☐ Pipeline comparison: 'dvc metrics diff' shows metric improvement between two pipeline versions

TOPIC 8 — Great Expectations: Data Quality Testing · *Week 6–7 · Automated data contracts for production AI pipelines*

Great Expectations is the leading data quality testing framework. You define 'expectations' — assertions about your data — and Great Expectations validates every dataset against them. When a data quality check fails, the pipeline stops before bad data corrupts your AI system. Think of it as pytest for data.

8.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Course	Great Expectations Tutorial — Complete Data Quality Guide	Great Expectations (official channel)	YouTube → Great Expectations official YouTube → Search 'Great Expectations Tutorial Complete Data Quality Guide'	Official GE tutorial. Watch the full intro series — covers core concepts and hands-on setup.
2	Video	Great Expectations with Pandas — Data Validation for ML Pipelines	Krish Naik	YouTube → Search 'Great Expectations Pandas Data Validation ML Pipelines Krish Naik'	Krish Naik applies GE to a real ML dataset — validates before training. Follow along.
3	Video	Data Contracts — How to Enforce Data Quality Between Teams	Zach Wilson (Data with Zach)	YouTube → Search 'Data Contracts Enforce Data Quality Between Teams Zach Wilson'	Data contracts are the concept; GE is the implementation. Watch this first for context.
4	Video	Great Expectations with Airflow — Data Validation in DAGs	Marc Lamberti	YouTube → Search 'Great Expectations Airflow Data Validation DAGs Marc Lamberti'	Integrate GE checks into Airflow DAGs — block tasks if data quality fails.
5	Video	Great Expectations Cloud — Managed Data Quality Platform	Great Expectations (official channel)	YouTube → GE official YouTube → Search 'Great Expectations Cloud Managed Data Quality Platform'	GE Cloud for teams: central expectation storage, data docs, and alerting.
6	Video	Soda Core vs Great Expectations — Data Quality Tool Comparison	Seattle Data Guy	YouTube → Search 'Soda Core vs Great Expectations Data Quality Tool Comparison Seattle Data Guy'	Compare GE with Soda Core — understand when each tool is the better fit.
7	Docs	Great Expectations Documentation — Quickstart Guide	Great Expectations Team	docs.greatexpectations.io — complete the Quickstart Guide (30 minutes)	Excellent quickstart. Complete it before watching any tutorials — creates your first expectation suite.

8.2 — Expectations for AI Data Pipelines

The Core Concept — Expectations as Data Contracts

```
import great_expectations as gx
import pandas as pd

context = gx.get_context()

# Create a validator for your AI document dataset
validator = context.sources.pandas_default.read_dataframe(
    df=pd.read_parquet('data/documents.parquet')
)

# ── SCHEMA EXPECTATIONS ──────────────────────────────────────────────
validator.expect_table_columns_to_match_ordered_list(
    ['doc_id', 'content', 'token_count', 'org_id', 'created_at']
)

# ── COMPLETENESS EXPECTATIONS ────────────────────────────────────────
validator.expect_column_values_to_not_be_null('doc_id')
validator.expect_column_values_to_not_be_null('content')
validator.expect_column_proportion_of_unique_values_to_be_between(
    'doc_id', min_value=0.999   # 99.9%+ unique IDs
)

# ── RANGE EXPECTATIONS ───────────────────────────────────────────────
validator.expect_column_values_to_be_between(
    'token_count', min_value=10, max_value=100000
)
validator.expect_column_mean_to_be_between(
    'token_count', min_value=200, max_value=5000   # avg doc length sanity check
)

# ── DISTRIBUTION EXPECTATIONS ────────────────────────────────────────
validator.expect_column_values_to_be_in_set(
    'language', ['en', 'hi', 'ta', 'te', 'kn', 'ml']
)
validator.expect_column_pair_values_a_to_be_greater_than_b(
    'updated_at', 'created_at'  # updated_at must be >= created_at
)

# Run validation — raises DataContextError if any expectation fails
results = validator.validate()
if not results['success']:
    raise ValueError(f'Data quality check FAILED: {results.statistics}')
    # Pipeline stops — bad data never reaches your LLM
```

Critical Expectations for AI Document Pipelines

No empty documents	expect_column_values_to_not_be_null('content') AND expect_column_value_lengths_to_be_between('content', 50, None)

No massive documents	expect_column_values_to_be_between('token_count', 10, 100000) — prevent context overflow
No duplicate chunks	expect_column_proportion_of_unique_values_to_be_between('chunk_id', 0.999) — prevent duplicate embeddings
Valid embedding dimensions	expect_column_values_to_have_length('embedding', 1536) — text-embedding-3-small always 1536
No stale data	expect_column_max_to_be_between('created_at', today - 2days, today) — pipeline is actually fresh
Org ID always present	expect_column_values_to_not_be_null('org_id') — prevents data leakage between organisations
Token count distribution	expect_column_quantile_values_to_be_between('token_count', {0.95: 8000}) — 95% under LLM context limit
No injection characters	custom expectation: content does not contain prompt injection patterns

Add one custom expectation to every AI data pipeline: check for prompt injection patterns in the document content. Malicious users who can inject text into your document corpus can control your AI's responses. Great Expectations makes this check automatic and blocking.

8.3 — Stop Conditions

- ☐ Great Expectations project initialised — 'great_expectations init' creates the project structure
- ☐ Expectation suite with 10+ expectations for your AI document dataset
- ☐ Validation checkpoint: runs the full suite against a new data batch — generates HTML data docs
- ☐ Airflow integration: a GE validation task runs before the embedding task — blocks on failure
- ☐ Custom expectation: no prompt injection patterns in document content — validates correctly
- ☐ Alert: GE validation failure sends Slack message with which expectations failed and row counts
- ☐ Data docs: 'great_expectations docs build' generates HTML report — team can review in browser

🗓 Phase 7 — Week-by-Week Schedule (8 Weeks)

Phase 7 is the most practical phase in the roadmap. Every week you are building real pipeline infrastructure, not just learning tools. By Week 8, all tools work together in one unified AI data platform that would impress any senior data engineer.

Week	Topic Focus	Exact Videos to Watch	Hands-On Task	Done When...
Wk 1	ETL Fundamentals + Prefect Advanced	ETL vs ELT Difference When to Use — Zach Wilson Data Pipeline Design Patterns Batch Streaming — Zach Wilson Prefect 3.0 Full Tutorial Series — Prefect official (parts 1–3)	Design (on paper) your AI platform's full data flow Upgrade Phase 3 Prefect pipeline to Prefect 3.0 with Work Pools Create prefect.yaml deployment definition	Prefect 3.0 deployment defined in code; Work Pool running; old flow migrated
Wk 2	Prefect Advanced + Airflow Setup	Prefect 3.0 Tutorial (parts 4–6) — Prefect official Prefect Events Automations Event-Driven — Prefect official Airflow Tutorial Beginners Full Course — CodeWithYu	Event-driven Prefect flow: new S3 file triggers ingestion Install Airflow locally via Astro CLI	Prefect event trigger working; Airflow UI accessible at localhost:8080
Wk 3	Airflow DAGs + TaskFlow	Airflow TaskFlow API Modern Python DAGs — Marc Lamberti Airflow Dynamic Task Mapping Parallel Processing — Marc Lamberti Airflow Best Practices Production-Grade DAGs — Astronomer	Rewrite Phase 3 ingestion pipeline as Airflow DAG with @task decorator Add dynamic task mapping: one task per document	Airflow DAG with 4 tasks working; dynamic mapping processes 100 docs in parallel
Wk 4	dbt — Staging to Marts	dbt Complete Beginner Advanced Course — dbt Labs (all) dbt Incremental Models Process Only New Data — Seattle Data Guy dbt with Airflow Orchestrate dbt Runs — Marc Lamberti	Initialise dbt project against your PostgreSQL database Build 3-layer model structure: staging → intermediate → marts Incremental model: fct_embedding_jobs	dbt project with 6+ models; incremental model verified; dbt tests pass
Wk 5	Advanced Pandas + Polars	Pandas Method Chaining Write Cleaner Code — Matt Harrison Polars Tutorial Fastest DataFrame Library — Liam Brannigan Polars Lazy API Process Data Larger Than RAM — Liam Brannigan	Rewrite 3 existing Pandas pipelines using method chaining + .pipe() Process a 1GB dataset with Polars — benchmark vs Pandas	Pipelines use method chaining; Polars benchmark shows >5x speedup on large dataset
Wk 6	Feast + DVC	Feature Stores Explained What They Are — W&B Feast Feature Store Complete Tutorial — Feast official DVC Official Tutorial Series — DVC official (parts 1–4)	Feast project: define document feature view + materialise to Redis DVC: track training dataset with S3 remote; create v1.0 and v1.1 versions	Feast feature retrieval <10ms; DVC checkout restores exact dataset version
Wk 7	Great Expectations + Integration	Great Expectations Tutorial Complete Guide — GE official Great Expectations Pandas Data Validation — Krish Naik Great Expectations Airflow Data Validation DAGs — Marc Lamberti	GE expectation suite with 10+ expectations for document dataset Integrate GE checkpoint into Airflow DAG — blocks on validation failure	GE validation runs in Airflow; data docs generated; failure sends Slack alert

Week	Topic Focus	Exact Videos to Watch	Hands-On Task	Done When...
Wk 8	Capstone — Full Data Platform Integration	Review: all pipeline orchestration docs Review: dbt + Airflow integration patterns	Connect all tools: • Airflow orchestrates the full pipeline • dbt transforms raw data • GE validates at every stage • DVC versions datasets • Feast serves features • Prefect handles event-driven re-embedding Write README with architecture diagram	One 'docker compose up' starts the entire data platform; README with lineage diagram

⍰ Phase 7 Capstone — Production AI Data Platform

This is Portfolio Project #7 — the data engineering backbone of your entire AI system. It combines every Phase 7 tool into a coherent, production-grade AI data platform that any data engineering team would recognise as professional infrastructure. This project demonstrates you can build and maintain the data layer that keeps AI systems reliable over time.

Full Project Specification

Project Name	ai-data-platform (GitHub repo name)
Orchestration	Apache Airflow (primary, enterprise-grade) + Prefect (event-driven, reactive pipelines)
Transformations	dbt Core — staging → intermediate → marts for all AI platform data
Data Quality	Great Expectations — validation checkpoints at every pipeline stage
Dataset Versioning	DVC — all training datasets, fine-tuning data, and embedding models tracked in S3
Feature Store	Feast with Redis online store — document features served <10ms at inference time
Processing	Polars for large file processing (>500MB); Pandas + .pipe() for <500MB
Storage	PostgreSQL (metadata, dbt output), S3/R2 (raw docs, DVC store, Parquet), Qdrant (vectors)
Monitoring	Airflow SLA alerts, GE data docs, Prefect Cloud dashboard, dbt test results
Deployment	'docker compose up' starts Airflow + PostgreSQL + Redis + Qdrant + dbt + GE

The Full Data Flow — Traced End to End

```
SOURCE: User uploads document to your AI SaaS (Phase 1-5)
    ↓
EVENT: S3 upload event triggers Prefect flow (near-real-time path)
    OR
SCHEDULE: Airflow DAG runs every 30 minutes (batch path)
    ↓
STAGE 1: Extract
  Airflow: fetch_new_documents() → pulls from PostgreSQL/S3
  Great Expectations: validate raw document schema
  BLOCKS if: null doc_id, empty content, suspicious token count
    ↓
STAGE 2: Transform
  Airflow: parse_documents() → PyMuPDF/python-docx
  Airflow: chunk_documents() → sliding window (512 tok, 50 overlap)
  dbt: stg_documents, stg_document_chunks (staging models)
  Great Expectations: validate chunks (no duplicates, correct lengths)
```

```
    ↓
STAGE 3: Embed
  Airflow: embed_chunks() with dynamic task mapping (one task per chunk)
  Prefect concurrency limit: max 20 concurrent OpenAI embedding calls
  DVC: checkpoint embedding batch as versioned artifact
    ↓
STAGE 4: Store
  Airflow: store_in_qdrant() → upsert to Qdrant vector DB
  dbt: fct_embedding_jobs, fct_rag_queries (incremental mart models)
  Feast: materialise document features to Redis online store
    ↓
STAGE 5: Monitor
  dbt test: run schema + relationship tests on mart tables
  GE checkpoint: validate final embedding job output
  Airflow SLA: alert if full pipeline takes >30 minutes
    ↓
RESULT: Fresh, validated, versioned document available in RAG within 30 minutes of
upload
```

Required Features

Airflow DAG Requirements

- ai_document_pipeline: the main 5-stage pipeline — hourly schedule, max_active_runs=1
- Dynamic task mapping: @task.expand() — one embedding task per document chunk
- S3 sensor: waits for files in the ingestion S3 bucket before processing
- SLA miss: Slack alert if pipeline hasn't completed in 30 minutes
- dbt_transformation: a separate DAG that runs 'dbt run + dbt test' after ingestion

dbt Model Requirements

- Staging: stg_documents, stg_document_chunks, stg_llm_api_calls, stg_embedding_jobs
- Intermediate: int_chunks_enriched (join chunks + document metadata)
- Marts: fct_embedding_jobs (incremental), fct_rag_queries, fct_ai_cost_daily
- Tests: not_null + unique on all primary keys; relationship tests on all foreign keys
- Docs: 'dbt docs serve' shows complete lineage graph — visible in browser

GitHub README Requirements

- Architecture diagram: full data flow from document upload to RAG availability
- Data lineage screenshot: dbt docs lineage graph showing all model dependencies
- Airflow DAG screenshot: the main pipeline with all task dependencies visible
- GE data docs screenshot: showing validation results with pass/fail statistics
- 'docker compose up' starts the complete platform in <2 minutes
- Pipeline SLA: documents available in RAG within 30 minutes of upload — benchmarked

Estimated Time Investment

Week 1 (ETL Fundamentals + Prefect Advanced)	~16 hrs — 8 hrs videos + 8 hrs upgrading Phase 3 pipelines
Weeks 2–3 (Airflow DAGs)	~22 hrs — 8 hrs tutorials + 14 hrs building and testing DAGs
Week 4 (dbt)	~18 hrs — 8 hrs course + 10 hrs building dbt models
Week 5 (Pandas Advanced + Polars)	~14 hrs — 6 hrs videos + 8 hrs refactoring pipelines
Week 6 (Feast + DVC)	~16 hrs — 8 hrs videos + 8 hrs hands-on setup
Weeks 7–8 (GE + Capstone Integration)	~20 hrs — 6 hrs tutorials + 14 hrs integration + documentation
Total Phase 7	~106 hours over 8 weeks ≈ 3 hrs/weekday + 5 hrs/weekend

Phase 7 Completion Check: You have completed Phase 7 when: (1) 'docker compose up' starts the full data platform, (2) uploading a new document results in it being searchable in RAG within 30 minutes, (3) the Airflow UI shows the full pipeline with all task dependencies, (4) dbt docs generates a complete lineage graph, and (5) a Great Expectations validation failure actually blocks the pipeline. This project proves you can build production-grade AI data infrastructure.

Phase 7's most common mistake: treating data tools as isolated islands. Airflow, dbt, GE, DVC, and Feast are most powerful when they work together — Airflow triggers dbt, dbt output feeds GE checks, GE failures stop the Airflow task, and DVC versions the output of the whole thing. Build the integration from week 1, not as an afterthought.